

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

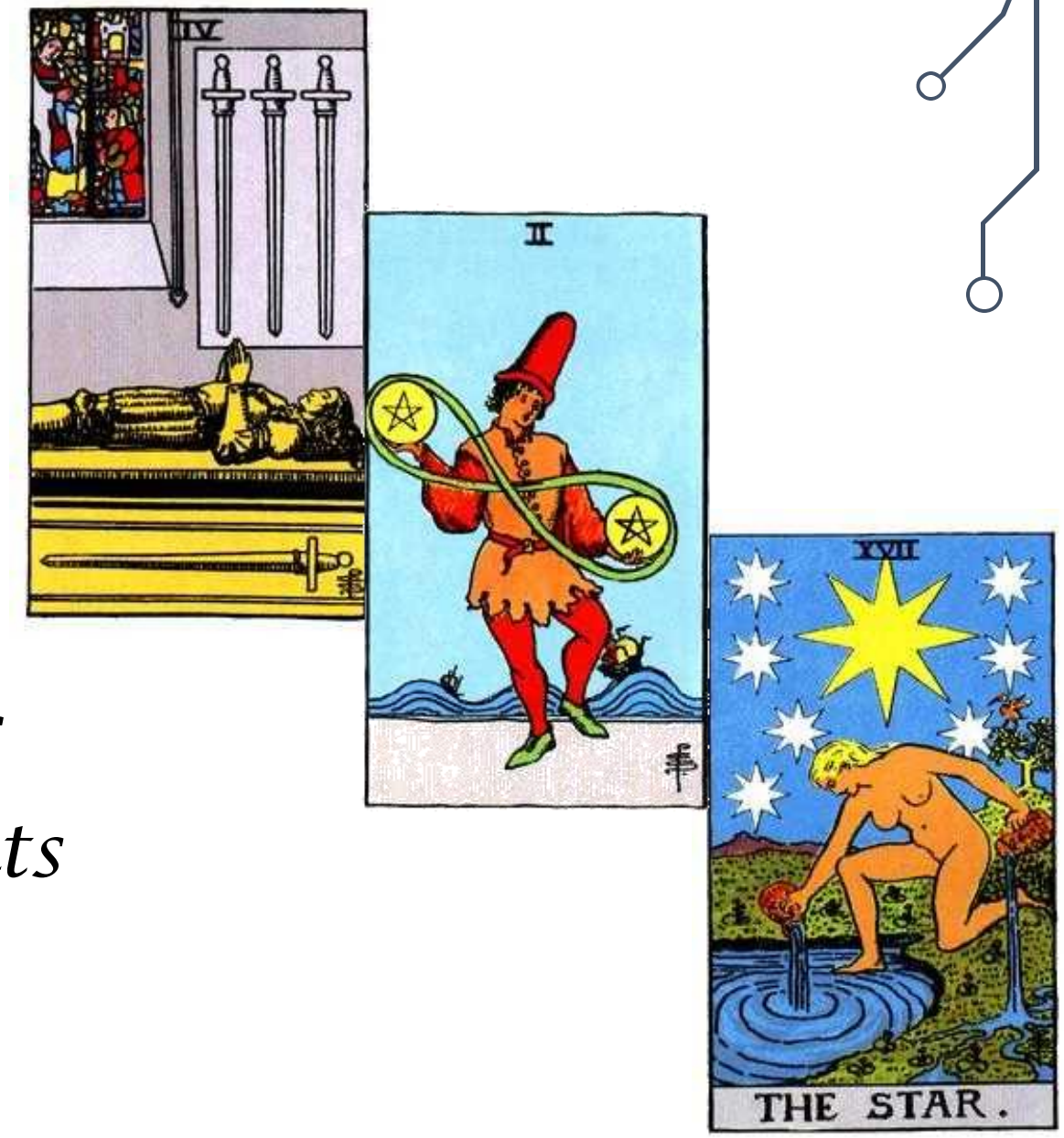
There's a sign on the wall,
but she wants to be sure

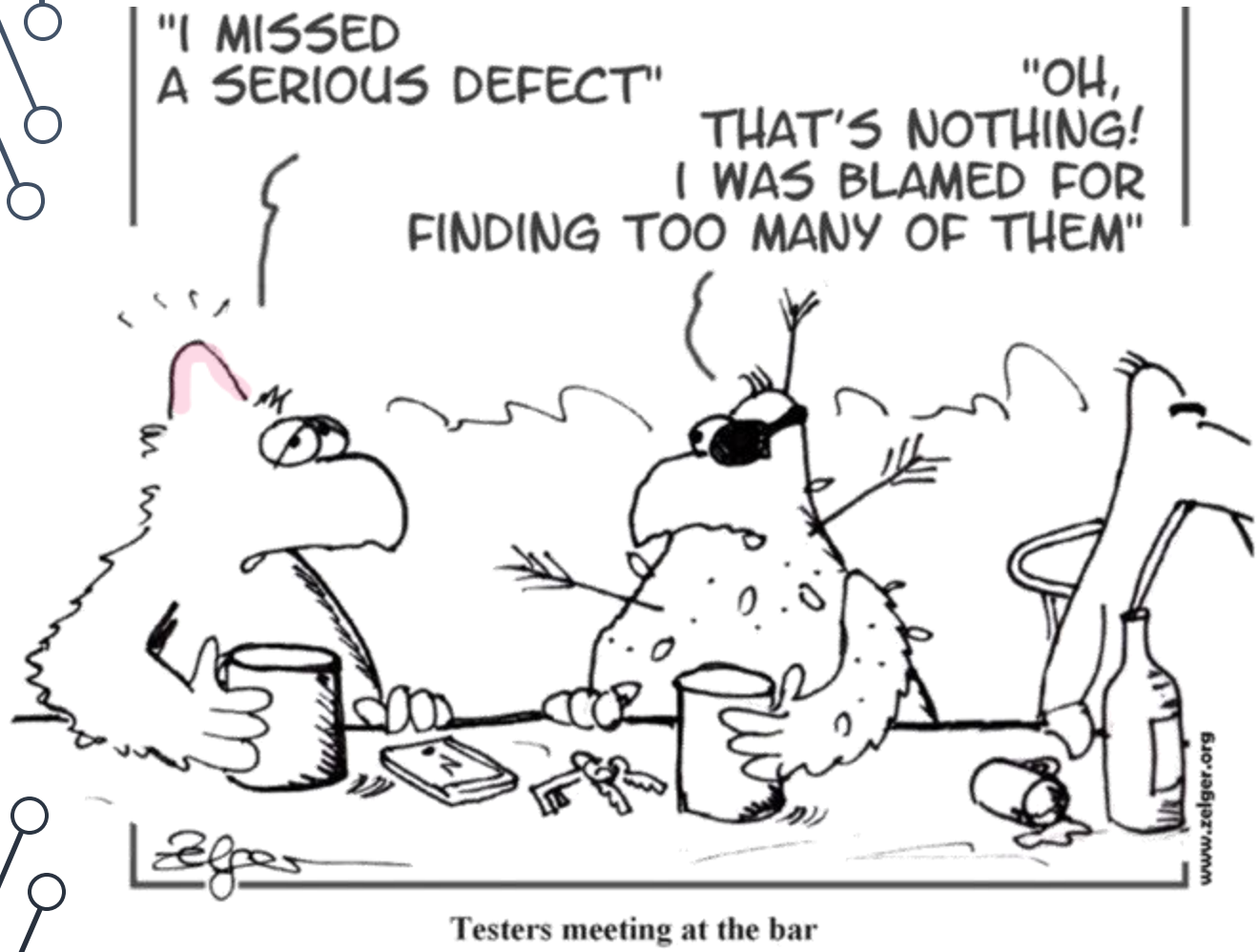
'Cause you know sometimes
words have two meanings

In a tree by the brook,
there's a songbird who sings

Sometimes all of our thoughts
are misgiven

- Chorus -





Testers meeting at the bar

CSC3003S ASD13 Dependable Systems

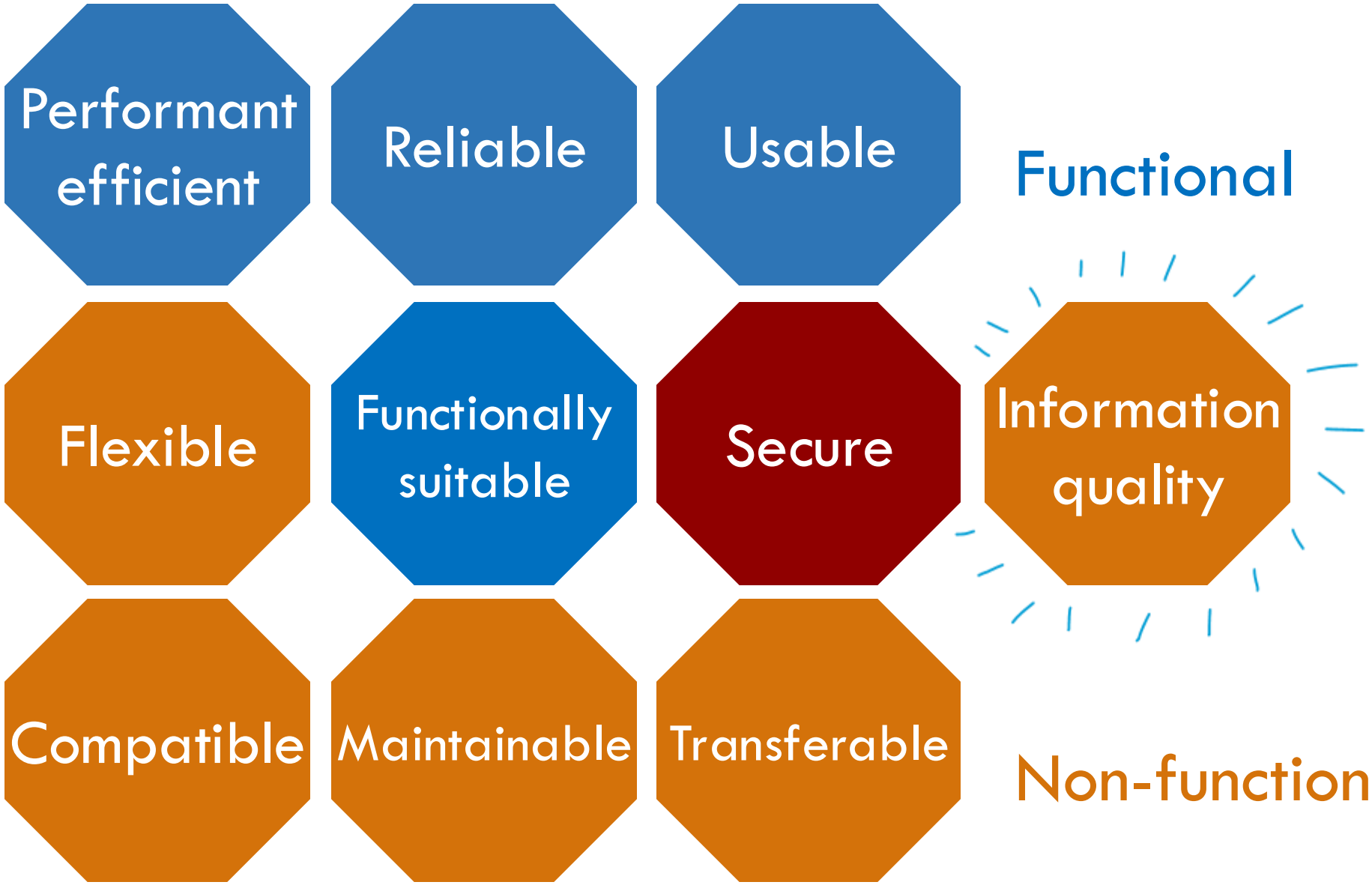
A. Software Quality (functional)

B. Software Quality (non-functional)

C. Quality-in-use

D. Software quality assurance

ISO 25010: Software quality



https://asq.org/quality-resources/software-quality
https://link.springer.com/content/pdf/10.1007/978-3-642-13911-6_15.pdf

ISO 25010 – 2011 Software quality metrics

Software quality								
Suitable	**Efficient**	**Compatible**	**Usable**	**Reliable**	**Secure**	**Maintainable**	**Flexible**	**Safe**
Complete	Time	Co-existence	recognizable	fault-free	confidential	modular	adaptable	operations
Correct	Resources	interoperable	learnable	availability	integrity	reusable	scalable	id risks
Appropriate	Capacity		user error	fault tolerant	non-repudiation	analysable	installable	fail-safe
			engagement	recoverability	accountability	modifiable	replaceable	hazard warn
			inclusivity		authenticity	testable		safe integration
			assistance		resistance			
			self-describe					

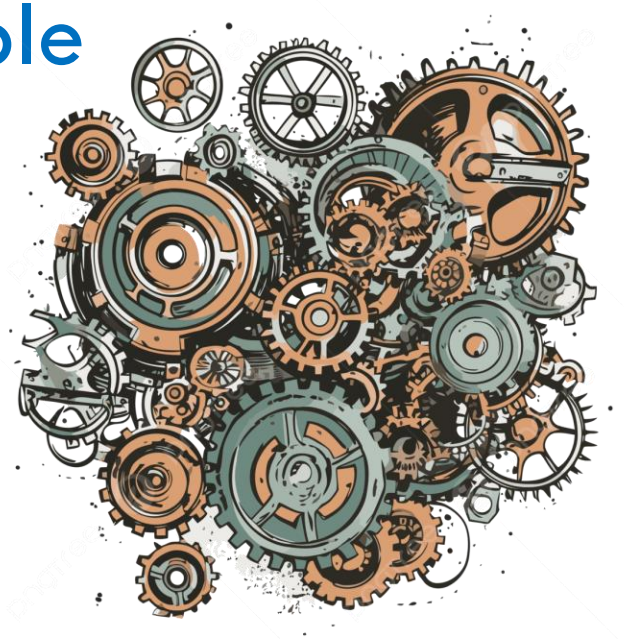
+ Information quality
Accurate
Usable
Secure

Lew et al., *Quality, Quality in Use, Actual Usability and User Experience as Key Drivers for Web Application Evaluation*, 2010

Quality aspects of information

• Complete	not too much (*the whole truth*)	Accurate
• Correct	not too little (*and nothing but the truth*)	
• Timely	correct timestamp (*robbing Peter to pay Paul*)	
• mathematical accuracy	correct calculations (*Excel sheets!*)	
• Legible	can be read/deciphered (*binary file formats*)	Usable
• Relevant	fits info demand/not outdated	
• Understandable	incl. assumptions	
• Comparable	both in time and against other data	
• Verified	has it been/can it be checked?	
• Trustworthy	believable (source?)	
• Legally compliant	violates no laws (*e.g.: VAT calculations*)	Secure
• Ethical	privacy/medical	
• Authorized	user allowed to access?	

Standards are made by people



- ISO 25010
  - Engineers
    - focus on models and processes
    - software as a machine
- Q aspects of info
  - Accountants
    - focus on information flows and procedures
    - software as an administrative system



- IT = information technology
  - but, information is implied in quality standards, not explicit
  - much like security was a few years ago

Performance/efficiency

Time
- Run time
- Latency
- Solution quality
- etc.

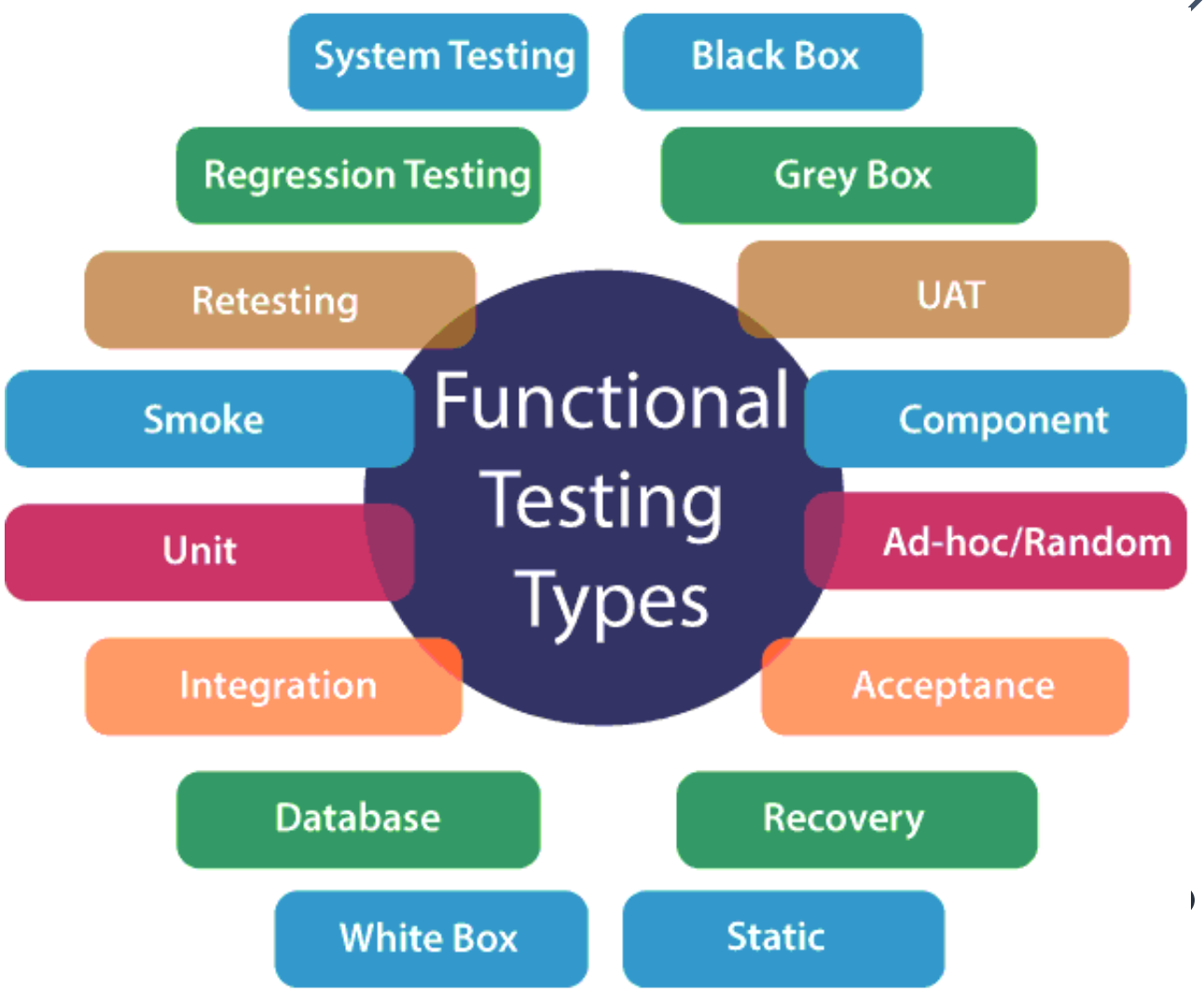
Resource
- Data storage needed
- Network usage
- CPU required
- etc.

Capacity
- How much data can it process



Functional suitability

- Complete
  - All needed features
- Correct
  - All implemented correct
- Appropriate
  - No unneeded features
  - No harmful features



Usability

Recognisable

- Use standard UI patterns

Learnable

Operable

Protection from user-error

- No data loss
- No idiotic showstoppers
  - Read: About face, chapter 12

UI aesthetics

Accessible

- Users with disabilities

Reliability

Available

- Uptime

Faultless

- no errors inside

Fault tolerant

- works under outside errors
  - outside hardware fails
  - outside software fails

Recoverable

- if fault:
  - recovery time
  - can it fully recover

Reliability: availability

Uptime *(measured in 'nines')*	
2: 99%	*3½ days per year*
3: 99.9%	*8 hours*
4: 99.99%	*1 hour*
5: 99.999%	*5 min*
6: 99.9999%	< 1 min

Availability: probability that a system is 'up'.

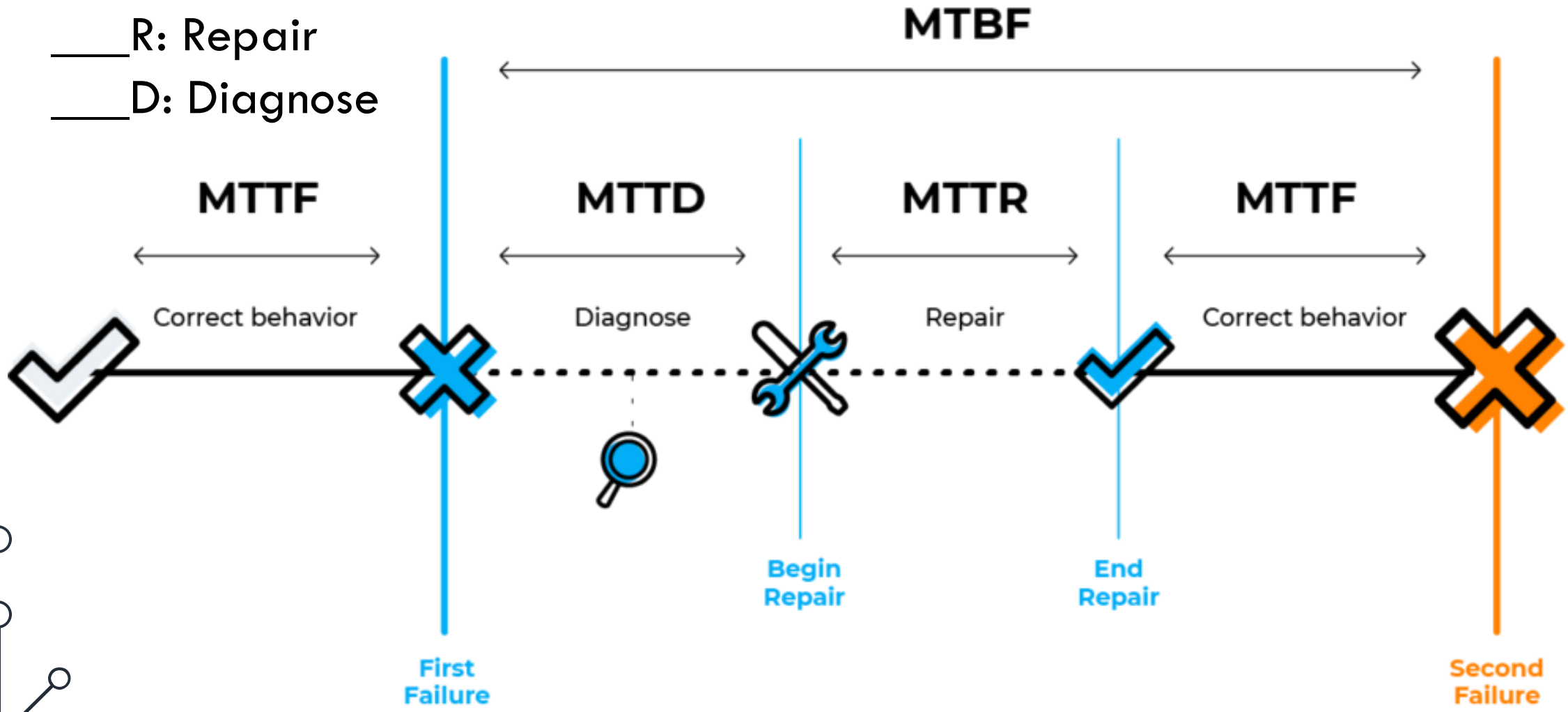
Reliability: availability

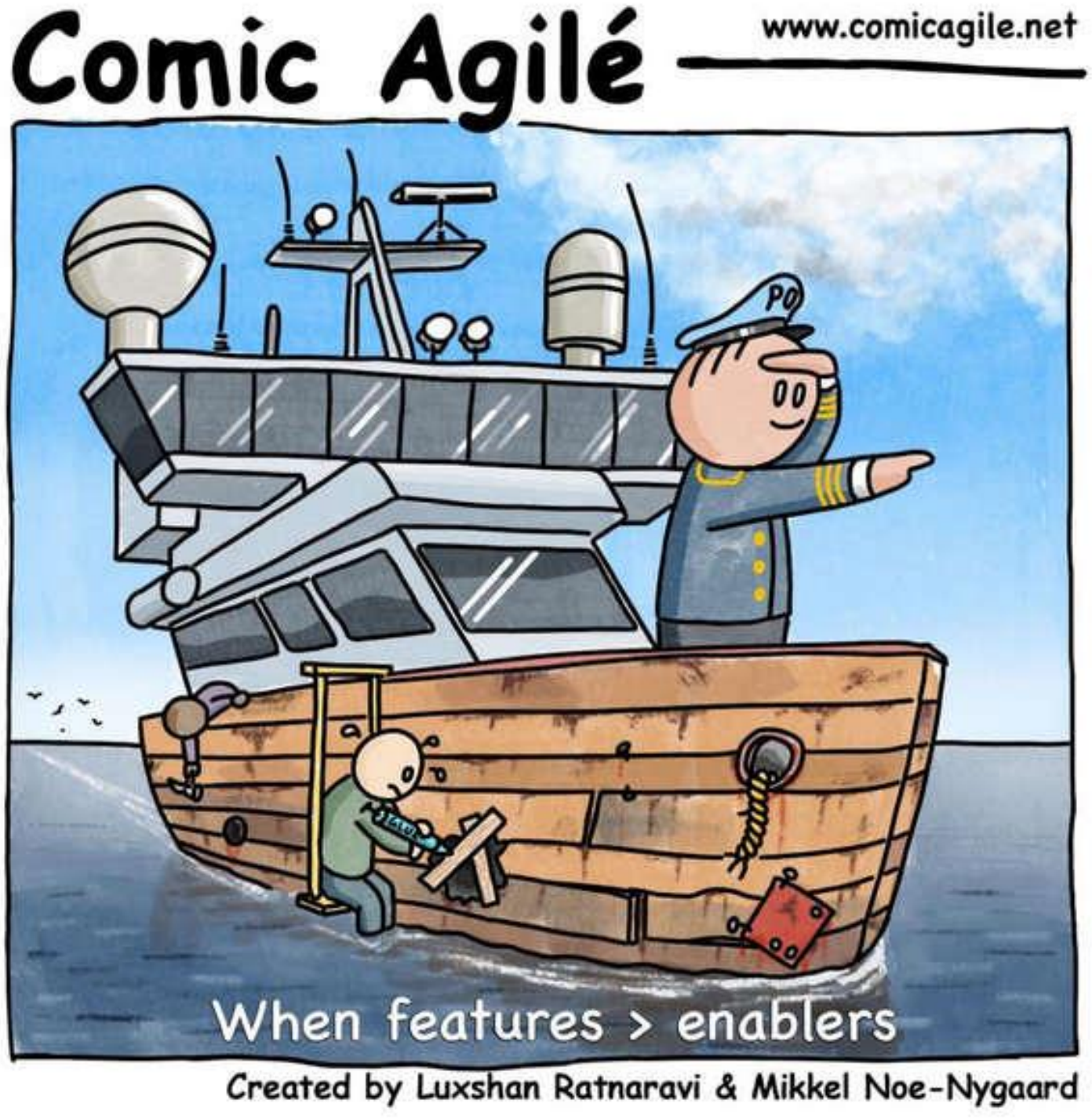
MTBF: Mean time between failures

MTTF: Mean time to failure

___R: Repair

___D: Diagnose





CSC3003S ASD13 Dependable Systems

A. Software Quality (functional)

B. Software Quality (non-functional)

C. Dependability Properties

D. Socio-technical Systems

Functional vs non-functional

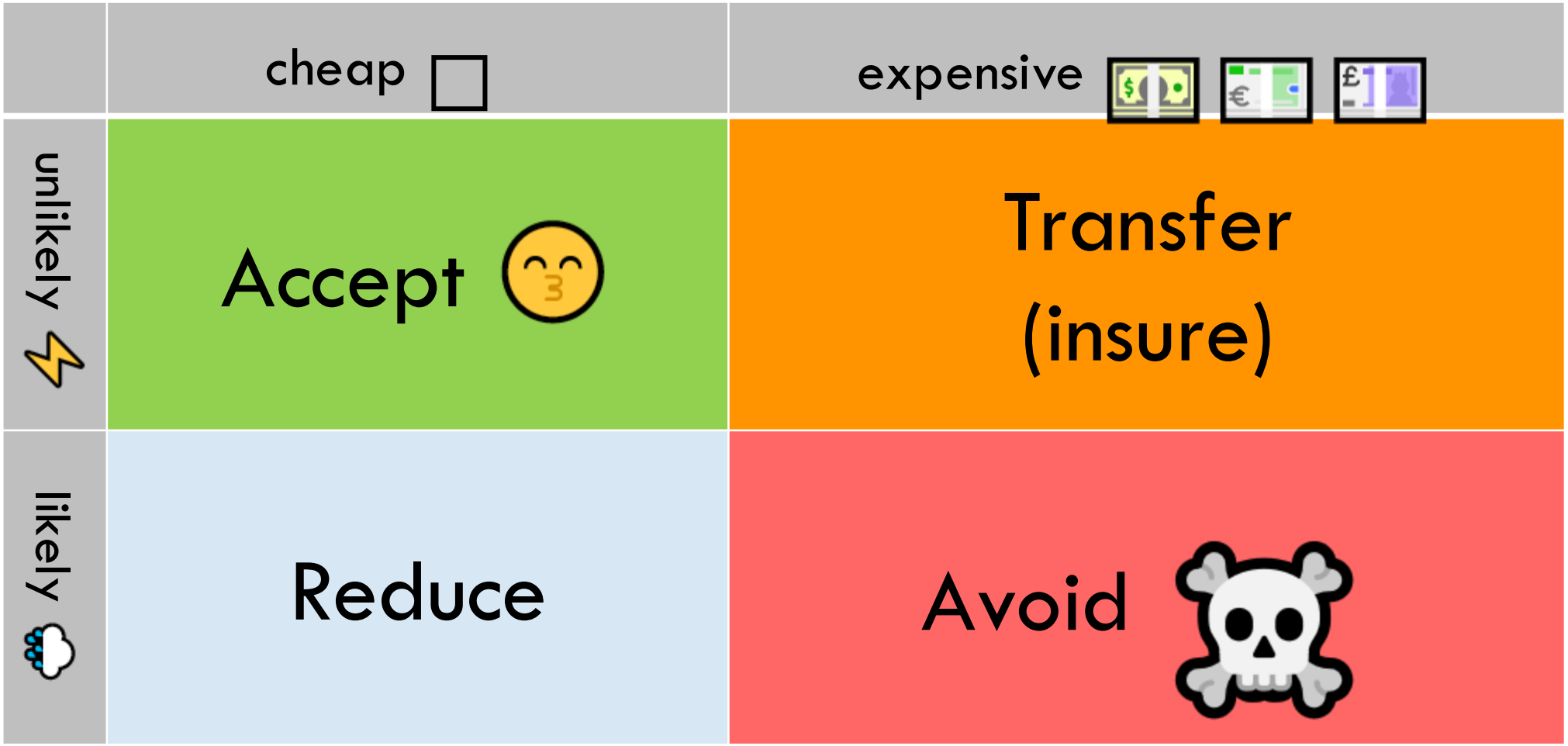
Distinction is somewhat blurry

Functional: directly affects users

Non-functional: users may not notice, but affects business risks

Security: most important quality metric

Why? => Risk = cost × likelihood



Maintainability

For ‘programmers’

- Modular
  - Few connections (low fan-out)
- Reusable
- Analysable
- Modifiable
- Testable



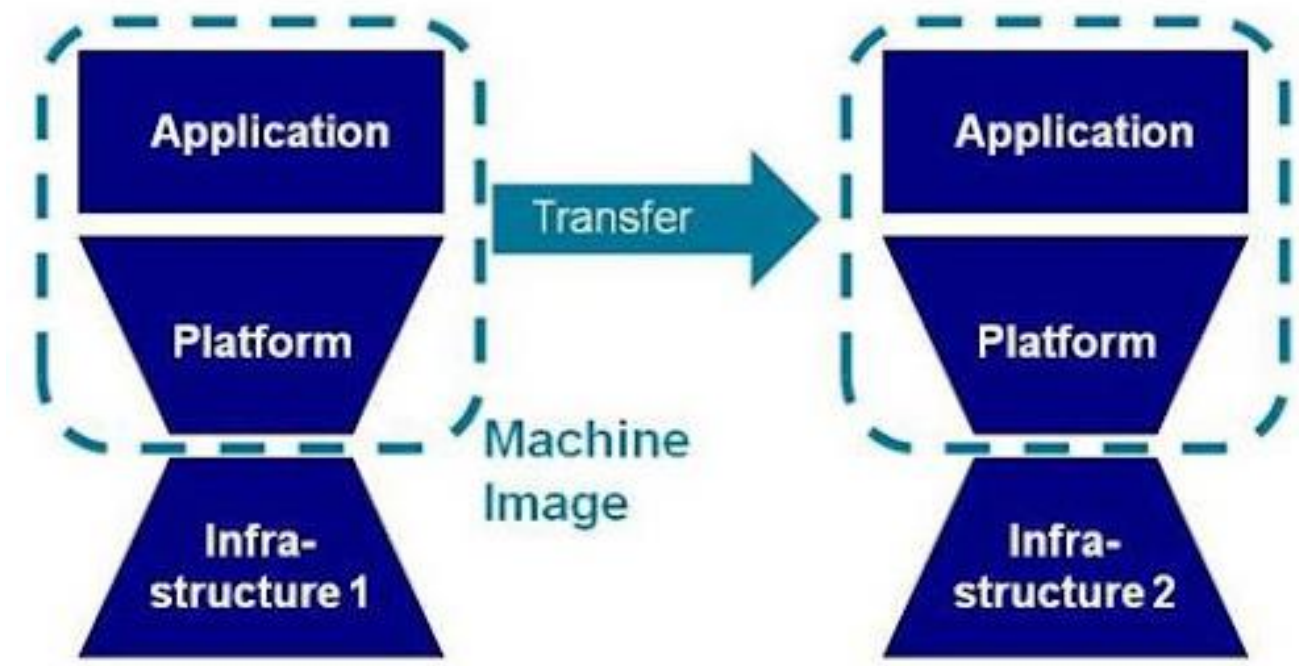
Flexibility

For ‘implementers’

- Adaptable, aka portable
- Scalable
- Installable
- Replaceable







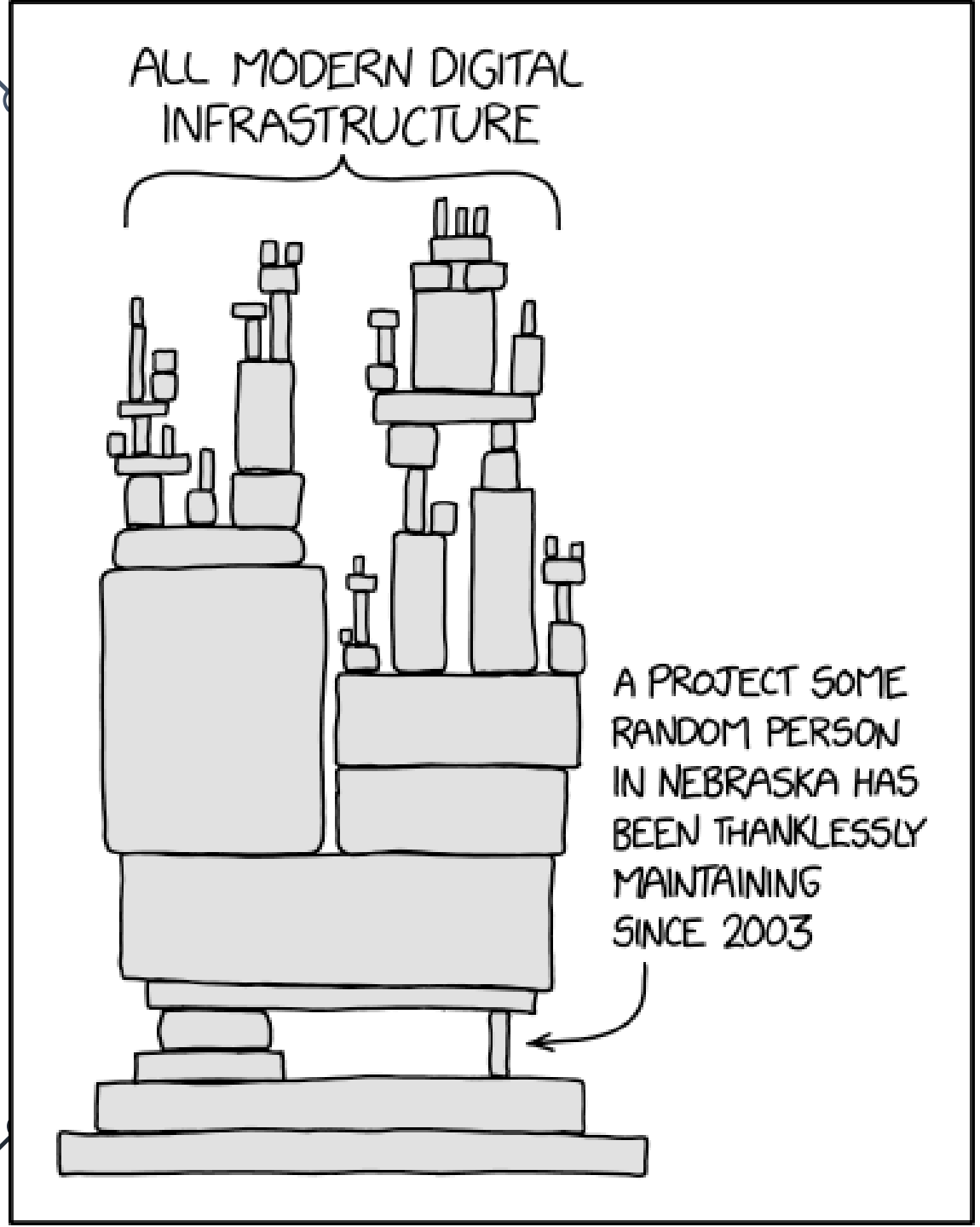
Safety

Not harmful

- Operations
  - Does not hurt people/organisation
- Id risks
  - Risks of harm are known
- Fail-safe
  - "*Plane does not crash*"
- Hazard warn
  - "*Format disk: are you sure?*"
- Safe integration

Compatible

- Co-existence
  - Does not misbehave with other software
- interoperable
  - Can exchange info with other software



CSC3003S ASD13
Dependable Systems

A. Software Quality (function)

B. Software Quality (non-functional)

C. Quality in use

D. Software quality assurance

Quality in use



Effectiveness
Efficiency
Satisfaction
Usefulness
Trust
Pleasure
Comfort
Freedom from risk
Economic risk mitigation
Health and safety risk mitigation
Environmental risk mitigation
Context coverage
Context completeness
Flexibility

← Outside quality perspective
The user's view

Software quality								
Suitable	**Efficient**	**Compatible**	**Usable**	**Reliable**	**Secure**	**Maintainable**	**Flexible**	**Safe**
Complete	Time	Co-existence	recognizable	fault-free	confidential	modular	adaptable	operations
Correct	Resources	interoperable	learnable	availability	integrity	reusable	scalable	id risks
Appropriate	Capacity		user error	fault tolerant	non-repudiation	analysable	installable	fail-safe
			engagement	recoverability	accountability	modifiable	replaceable	hazard warn
			inclusivity		authenticity	testable		safe integration
			assistance		resistance			
			self-describe					

← Inside quality perspective
The programmer's view

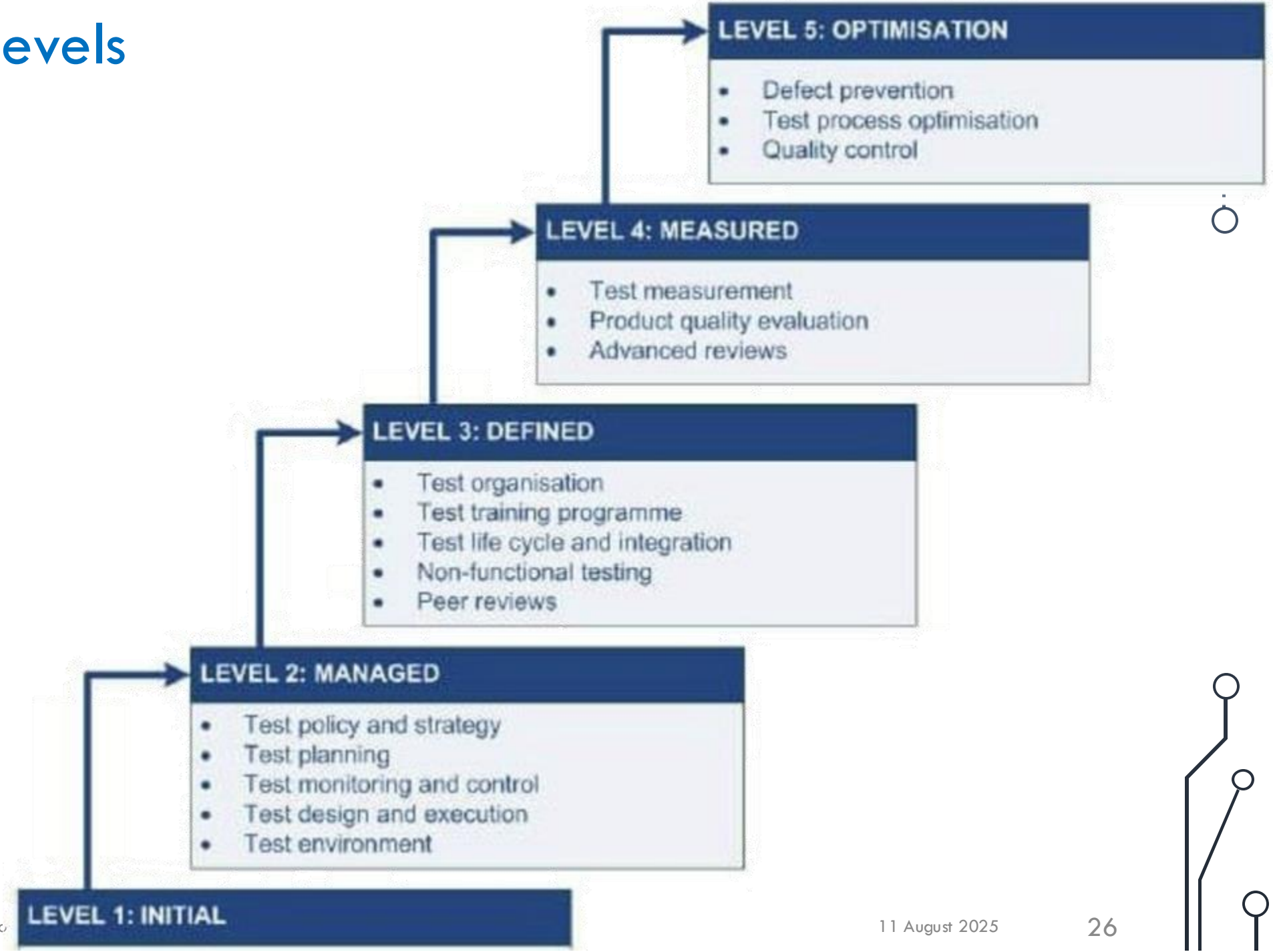
Quality control

- Reactive, tries to detect quality problems
- Testing
  - Unit tests
  - Automated tests
  - Functionality tests
  - Stress tests
  - Alpha/Beta tests
  - …
- Does not prescribe procedures and standardised documentation of work done

Quality assurance

- Preventative: puts procedures in place to assure quality (ISO 9000 series)
- Documentation of work done to see after the fact that procedures were followed
  - Checklists
- Useful for quality certification
- Requires a documented audit trail
- Formal assurance assumes a waterfall model
  - Quality is checked against documentation, esp. requirements + analysis
    - Each stage in waterfall has its own documents
    - W's: who, what, why, where, when
- Pure Agile assumes a design grows organically
  - Hard to certify, due to minimal documentation

Maturity levels



Dromley's quality model

- Every model (incl. Dromley) has different metrics
  - Different breakdown of "quality" into subcomponents
- We'll disregard that and just keep following ISO 25010
- Dromley's 5-step process
  - Pick high level quality metrics to evaluate
  - List components in system
  - Identify quality-carrying properties of components
  - Determine how property affects quality metric
  - Evaluate, identify weakness, use risk matrix to mitigate

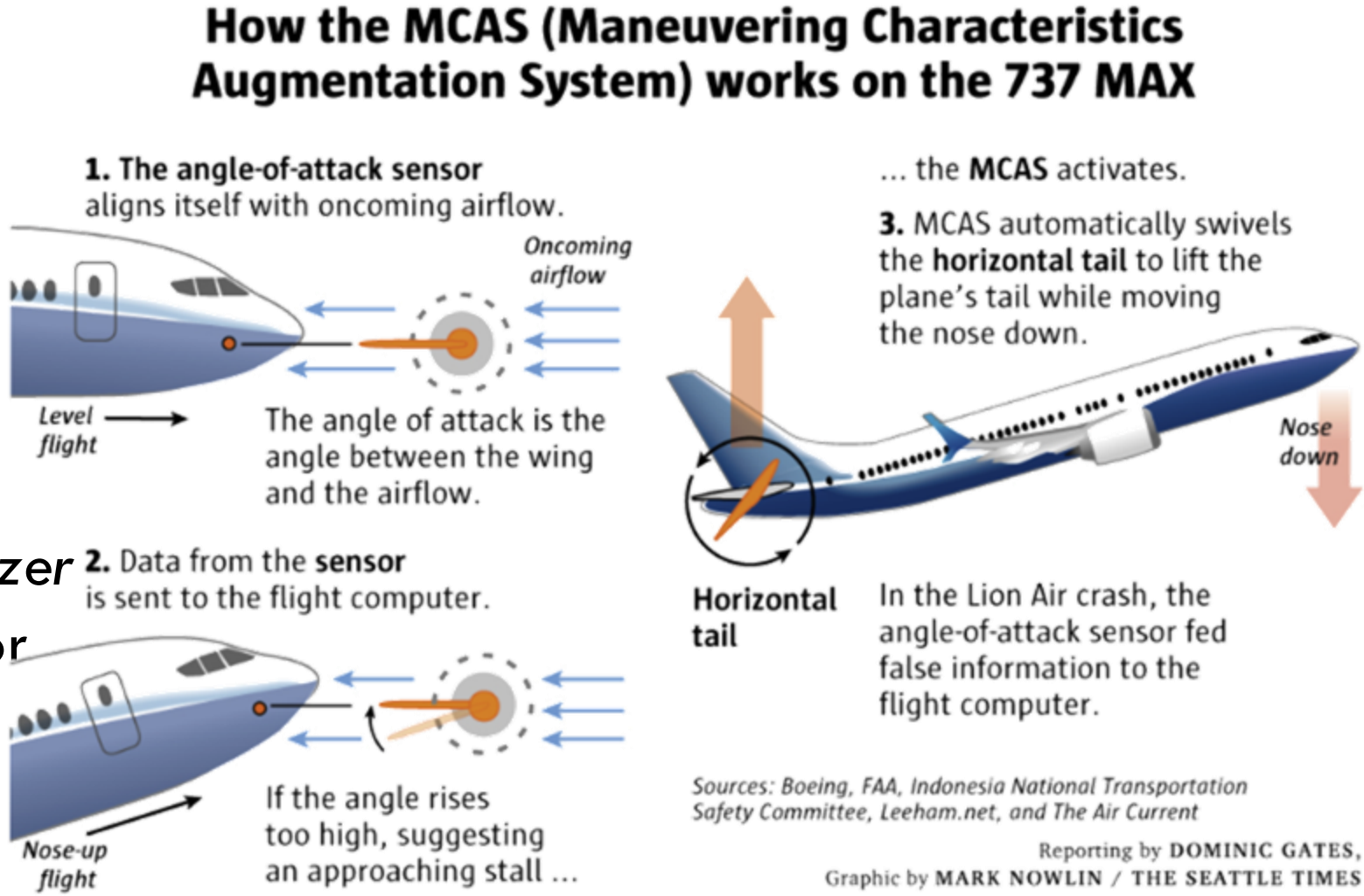
The cost of poor quality is exponential

- Errors in specification and development (communication and testing)
- Verification and validation issues
- Availability problems – system goes down
- System buggy
- Security problems – costly risks
  - Cyber attacks
- Design for recovery from failures and attacks without loss of data

Bad design

MCAS

- Solves trivial problem
- No training, just a bulletin
- Pilot has 3 secs to respond
- Has hidden cheat code
- But, restarts every 5 secs
- Not on checklist *runaway stabilizer*
- Depends on a single AoA sensor
- Bigger correction every restart
  - up to 90° nose down ↓
- Uses tail => huge lever
  - Pilot must use kgs then tons to overcome
- Can only be switched off manually
  - if you do, stabilisers are stuck



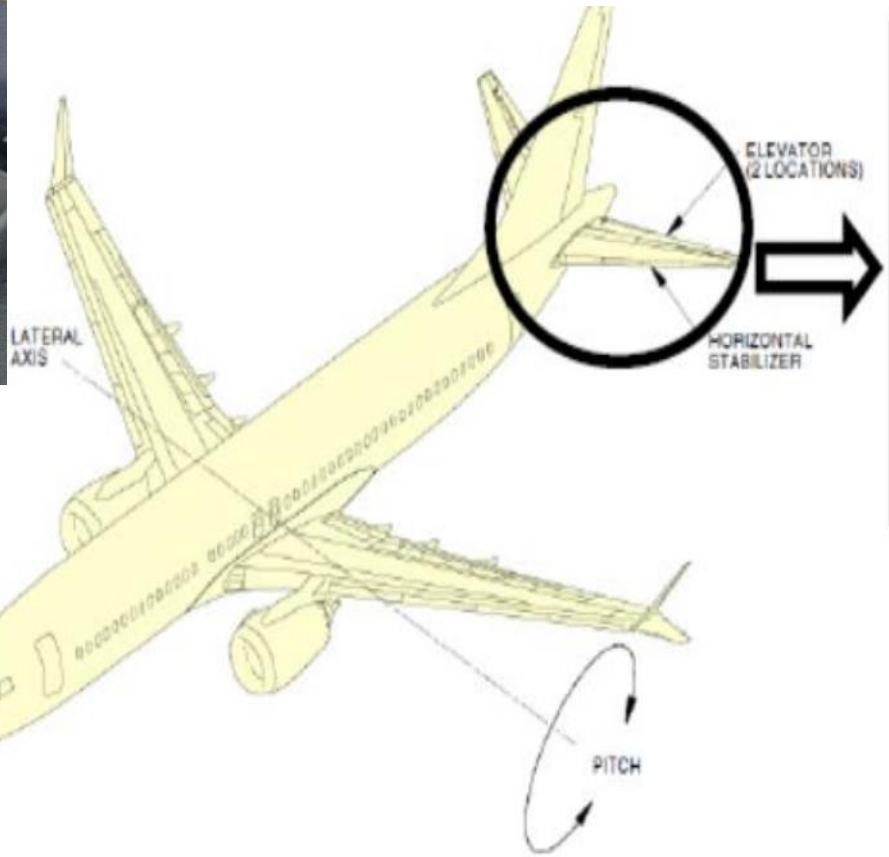
MCAS

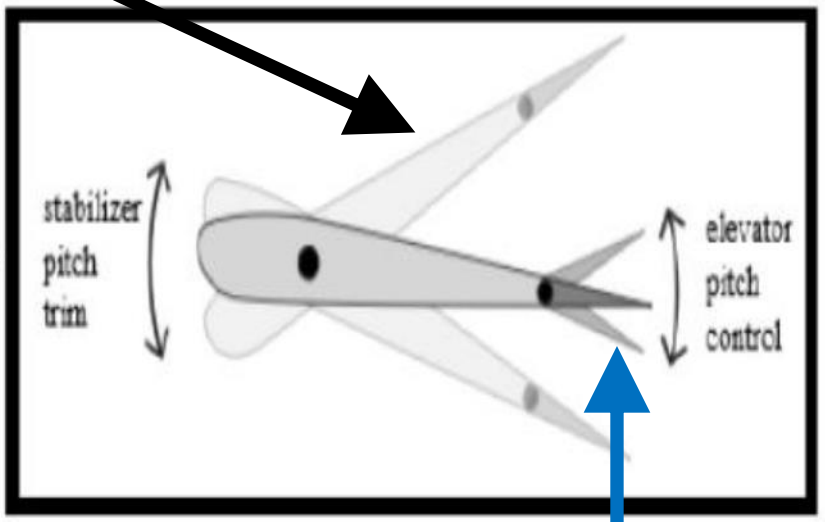


MCAS, increasing speed

trim switches, fixed slow speed

manual trim wheel, awkward





Pilot Joke

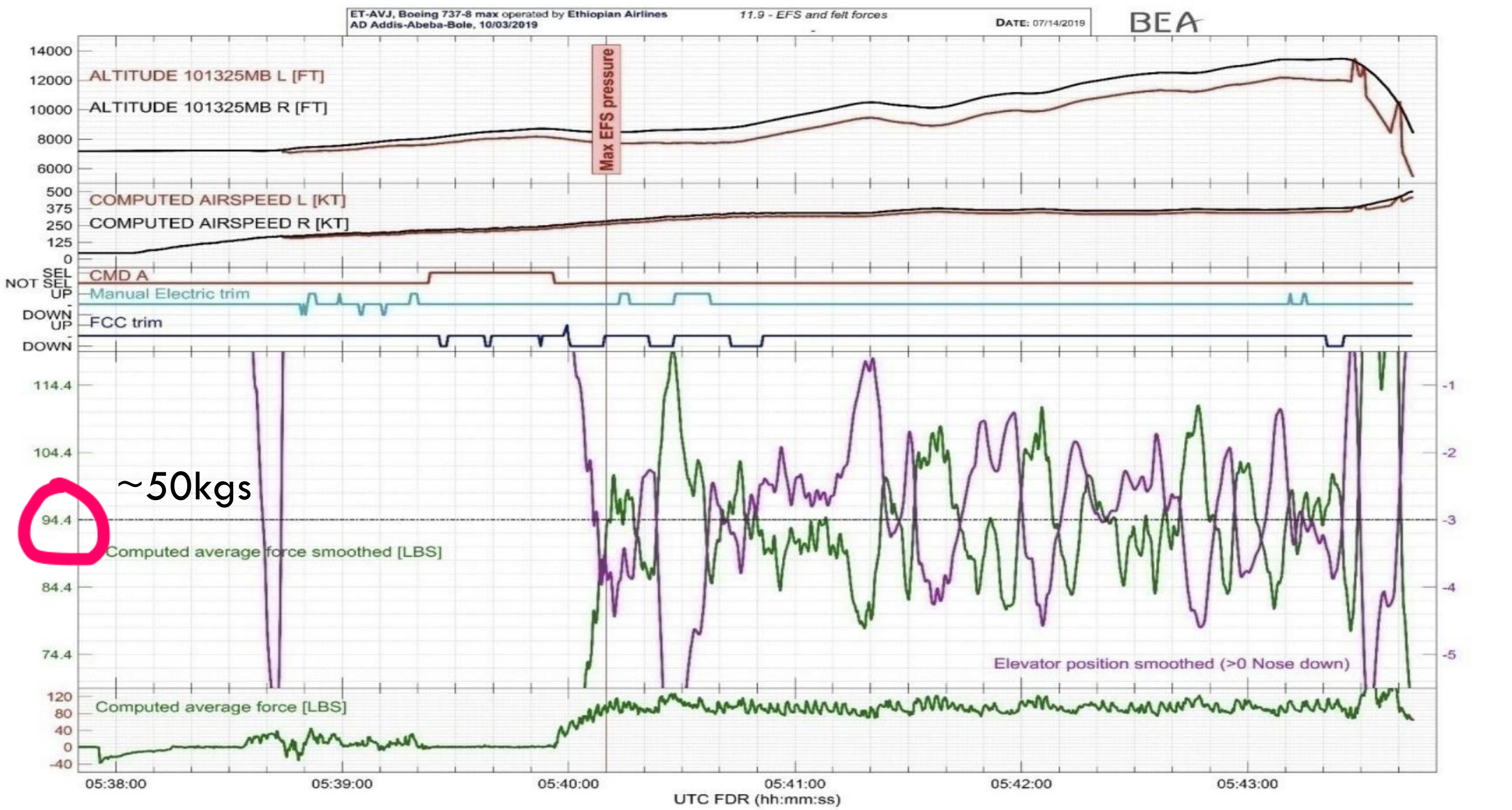


FIGURE 79: FELT FORCE AND ELEVATOR POSITION

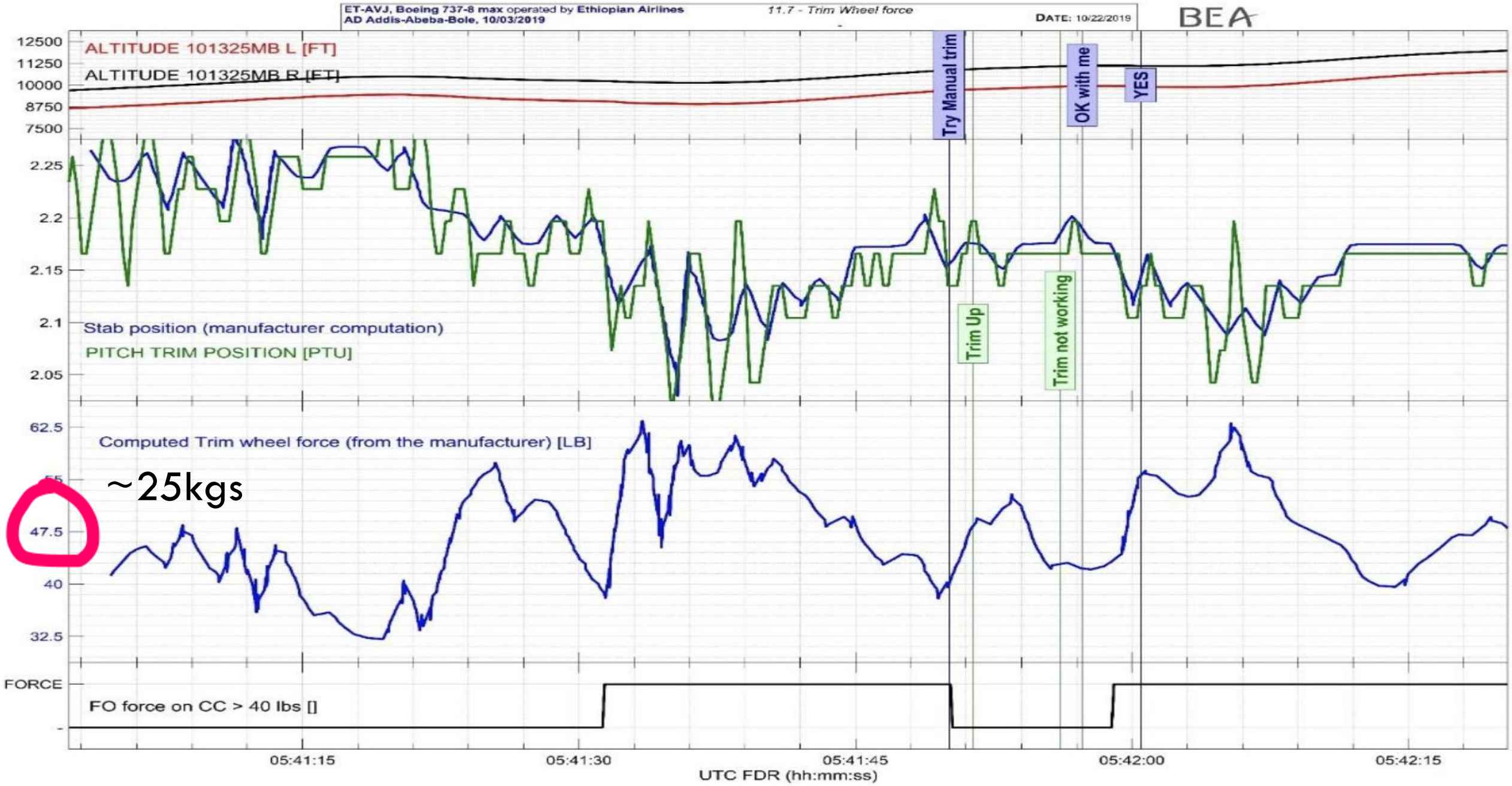


FIGURE 77: FORCE NEEDED TO MOVE THE TRIM WHEEL